SysMonitor

System Description

Abstract
This document describes the SysMonitor support system of the Eclipse Arrowhead 5.2.2 framework, as implemented in `ah5-SysMonitor-java-spring` with an independent local-cloud deployment definition in its own folder.

Contents

1 Overview **3**
1.1 Significant Prior Art . . . 4
1.2 How This System Is Meant to Be Used . . . 4
1.3 System functionalities and properties . . . 4
1.4 Important Delimitations . . . 4

2 Services **5**
2.1 Produced services . . . 5
2.2 Consumed services . . . 5

3 Security **6**
3.1 Security model . . . 6

4 References **6**

5 Revision History **7**
5.1 Amendments . . . 7
5.2 Quality Assurance . . . 7

1 Overview

This document describes the SysMonitor system. SysMonitor collects the native `sysMonitor` observations of all systems from the AMQP broker, stores their history and republishes them on the MQTT broker (`sysMonitorData`).



Figure 1: SysML-style block view of SysMonitor with its produced and consumed services.

The rest of this document is organized as follows. In Section 1.1, we reference major prior art capabilities of the system. In Section 1.2, we describe the intended usage of the system. In Section 1.3, we describe fundamental properties provided by the system. In Section 1.4, we describe delimitations of capabilities of the system. In Section 2, we describe the abstract service functions consumed or produced by the system. In Section 3, we describe the security capabilities of the system.

1.1 Significant Prior Art

New in this workspace; native Java port of the former Node.js implementation (sources in `Implementation` kept for rollback).

1.2 How This System Is Meant to Be Used

SysMonitor orchestrates the `sysMonitor` providers, binds their routing keys per subscription, stores observations transactionally and forwards them. LocalCloudManager edits subscriptions and reads history; the Plant DescriptionEngine reads liveness.

1.3 System functionalities and properties

1.3.1 Functional properties of the system

- Orchestration of sysMonitor providers and AMQP binding per subscription.
- Validation, deduplication and storage with 30-day retention.
- Transactional outbox to MQTT (QoS 1).
- History queries with authorization.

1.3.2 Configuration of system properties

The system is configured with environment variables. The variables that shape its behaviour are:

Property	Default	Meaning
`PORT`	8475	HTTPS port.
`AMQP_HOST` / `AMQP_PORT`	amqp-broker / 5671	Intake broker.
`MQTT_HOST` / `MQTT_PORT`	mqtt-broker / 8883	Output broker.
`SYSMONITOR_RETENTION_DAYS`	30	History retention.
`SYSMONITOR_DEFAULT_SUBSCRIPTION`	all	`all` or `none`.
`ORCHESTRATION_INTERVAL_SECONDS`	30	Provider re-orchestration.
`LOCAL_CLOUD`	TestCloud	Routing-key cloud segment.

Secrets (database, keystore and identity passwords, secret keys) have no usable defaults; the standalone deployment reads them from per-deployment Compose file secrets under `/run/secrets`.

1.3.3 Data stored by the system

Observations, outbox, subscriptions, AMQP bindings and audit (SQLite).

1.3.4 Non functional properties

AMQP deliveries are acknowledged only after persistence.

1.3.5 Stateful or stateless

Stateful.

1.4 Important Delimitations

History is not a long-term archive; duplicates on `sysMonitorData` are possible after a crash.

2 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

2.1 Produced services

Service	Version	Interfaces	SD	IDD
`sysMonitorManagement`	1.0.0	HTTP(S)	[1]	[2]
`sysMonitorHistory`	1.0.0	HTTP(S)	[3]	[4]
`sysMonitorData`	1.0.0	MQTT(S)	[5]	[6]
`sysMonitor`	1.0.0	AMQP(S)	[7]	[8]

All services use `generic_https` with `CERT_AUTH`; `sysMonitor` uses `generic_amqps`. The container also answers `GET /health` on a loopback-only plain-HTTP port for the container health check.

2.2 Consumed services

Provider	Service	Operations	Use	Condition
ServiceRegistry	serviceDiscovery	register, lookup	Registration; orchestrator lookup.	always
Authentication	identity	identity-login, identity-verify	Own token; caller verification.	always
DynamicService Orchestration	service Orchestration	pull	sysMonitor providers.	always
Consumer Authorization	authorization	verify	History authorization.	history
AMQP broker	sysMonitor (all providers)	echo, power, status, . . .	Intake.	subscriptions
MQTT broker	sysMonitorData	publish	Output.	always

Only Authentication's `identity` and the orchestrator are looked up in the ServiceRegistry; other services are reached through DynamicServiceOrchestration (results cached), and every such call needs a Consumer-Authorization grant.

3 Security

The standalone Java system requires HTTPS with mutual TLS and X.509 certificates following the Arrowhead system certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`, DNQ `sy`). There is no insecure runtime mode. Identity tokens are verified through Authentication and bound to the caller certificate. A separate loopback health endpoint is infrastructure only. Provision credentials and a valid CRL using the project-local environment template and Compose definition.

3.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates; MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; SQLite storage is local to this system's private volume. Passwords are read from Compose file secrets; certificate and private-key files are mounted individually.
- **System authentication:** mutual TLS with a system or operator certificate of the cloud and an identity token of the same system, verified with Authentication; declared identities are not accepted. Client certificates are checked against the CRL.
- **Authorization of produced services:** per operation by caller lists in the implementation and, where stated, by ConsumerAuthorization.
- Observations are accepted only if their identity matches the authorized routing key. Management only LocalCloudManager and Sysop.

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

4 References

[1] J. Delsing, "sysMonitorManagement – Service Description (SysMonitor)," Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorManagement-SD.pdf.

[2] ——, "sysMonitorManagement HTTP/TLS/JSON – Interface Design Description (SysMonitor)," Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorManagement-HTTP-IDD.pdf.

[3] ——, "sysMonitorHistory – Service Description (SysMonitor)," Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorHistory-SD.pdf.

[4] ——, "sysMonitorHistory HTTP/TLS/JSON – Interface Design Description (SysMonitor)," Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorHistory-HTTP-IDD.pdf.

[5] ——, "sysMonitorData – Service Description (SysMonitor)," Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorData-SD.pdf.

[6] ——, "sysMonitorData MQTT/TLS/JSON – Interface Design Description (SysMonitor)," Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorData-MQTT-IDD.pdf.

[7] ——, "sysMonitor – Service Description (SysMonitor)," Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitor-SD.pdf.

[8] ——, "sysMonitor AMQP/TLS/JSON – Interface Design Description (SysMonitor)," Eclipse Arrowhead, IDD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitor-AMQP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	